

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Nicollas Zoratto de Carvalho

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	4
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	5
4.4	Atividade 4 — Controle de acesso por papel — RBAC	6
4.5	Atividade 5 — Logs e auditoria	7
4.6	Atividade 6 — Upload e perfil de usuário	9
5	Considerações finais	10
6	Declaração de autoria	10

1 Introdução

O relatório apresenta o desenvolvimento e as entregas bimestrais referentes à disciplina de Introdução à Computação em Nuvem (ISW055). O objetivo central das atividades foi evoluir uma aplicação web — o **NicoFlix** (catálogo de filmes focada nos trabalhos de Tom Hanks) — partindo de um modelo monolítico inicial até uma arquitetura moderna distribuída em microsserviços em containers.

Ao longo do período, foram abordados conceitos fundamentais de desenvolvimento em nuvem, tais como Docker, orquestração de containers com DockerCompose, autenticação desacoplada, controle de acesso baseado em papéis (RBAC), serviço de auditoria e logs com Redis, e armazenamento de objetos (Object Storage) com MinIO.

A maioria desses assuntos confesso que não tinha conhecimento profundo acerca, principalmente Docker. Até agora tem sido uma das melhores formas de ensino que venho tendo, com ênfase no processo prático (mesmo que sendo com auxílio de I.A).

Este documento está organizado de forma a detalhar a metodologia utilizada para o versionamento e ambiente de execução, apresentando um quadro resumo das atividades e fichas individuais contendo as evidências de cada entrega, links para o repositório público no GitHub e os percalços técnicos enfrentados.

2 Metodologia

Todas as etapas do projeto **NicoFlix** foram estruturadas para execução containerizada em ambiente Linux e implantadas no ambiente do servidor da disciplina gerenciado via Portainer.

Em relação ao controle de versão (Git/GitHub), cumpre esclarecer a dinâmica de versionamento adotada:

1. **Atividades 1 a 3 (Reestruturação de Repositório):** Nas fases iniciais do projeto, devido a constantes reestruturações arquiteturais e conflitos complexos no controle de versão durante a separação do monolito para microsserviços (auth_service), optou-se por recriar o repositório no GitHub para manter uma base de código limpa. O código completo acumulado das Atividades 1, 2 e 3 foi consolidado no commit inicial cb6cb6c em **31/08/2026**. As implantações dessas etapas foram realizadas diretamente na stack do Portainer nos prazos correspondentes.
2. **Atividades 4 a 6 (Versionamento Contínuo):** A partir da quarta atividade, o versionamento seguiu o fluxo convencional no repositório público NicollasZoratto/NicoFlix, registrando as alterações através de commits auditáveis (3690909 e 462bc91).

Ou seja, sendo totalmente sincero, não possuo os commits das atividades 1, 2 e 3, pelo fato da dificuldade de merge (muitas alterações de arquivo, -force, arquivos novos). O que ocorreu foram remoções de repositórios antigos e criação de novos individualmente para as atividades propostas (Se reparar o commit da atividade 4 explicita isso). Contudo, os prazos de entrega via Portainer e desenvolvimento foram atendidas.

As evidências apresentadas neste relatório foram extraídas diretamente dos registros do GitHub, das configurações do docker-compose.yml e das inspeções das instâncias em execução no Portainer.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	—	não entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	31/08/2026 19:46	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	31/08/2026 19:46	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	31/08/2026 22:15	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	23/09/2026 20:05	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	05/10/2026 19:19	entregue com atraso

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	—
Situação	não entregue
Evidência	Não aplicável
Link	—

O que foi feito. Devido à ausência na aula em que a atividade de nivelamento foi proposta, esta atividade específica não foi desenvolvida individualmente no prazo inicial.

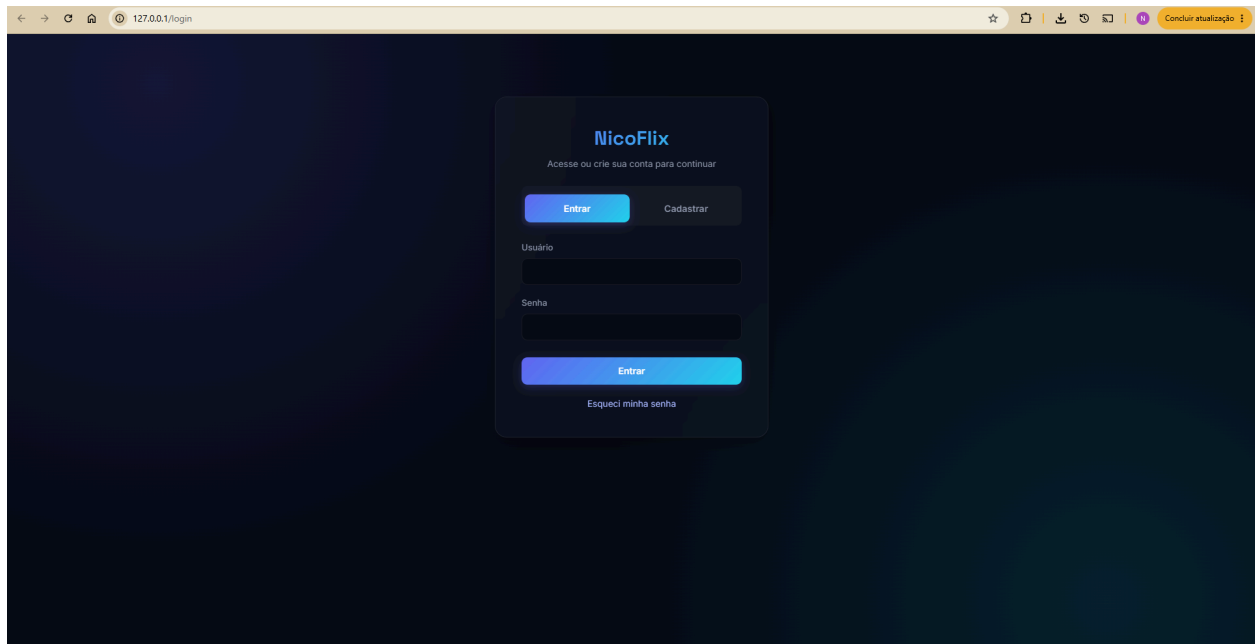


Figura 1: Atividade 1 — evidência da entrega (não realizada)

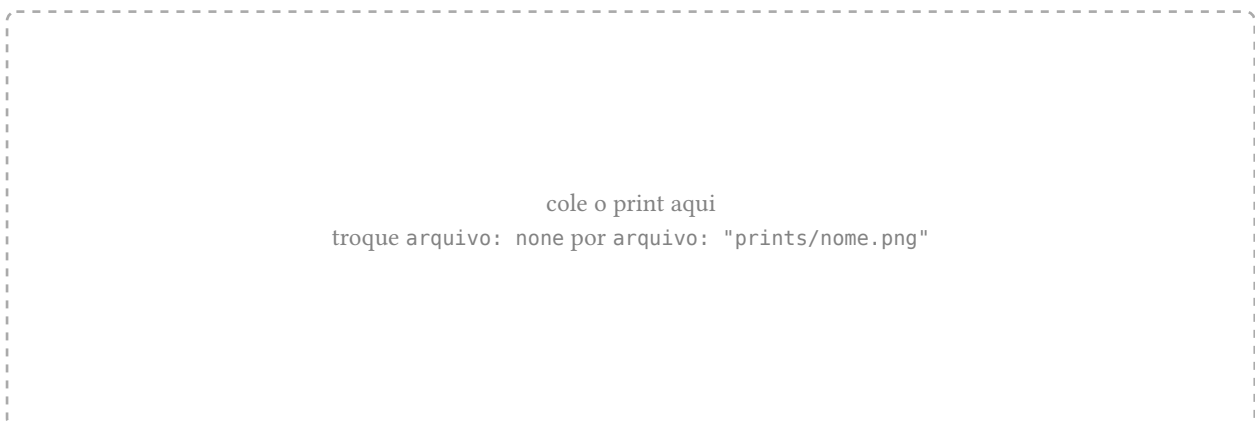


Figura 2: Atividade 1 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. Por não ter acompanhado a aula presencial, a introdução ao Flask foi compensada posteriormente durante o desenvolvimento da Atividade 2, na qual a estrutura do framework foi absorvida diretamente na construção da aplicação principal do catálogo de filmes.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	31/08/2026 19:46
Situação	entregue
Evidência	GitHub — commit cb6cb6c (Código incorporado no refatoramento inicial)
Link	https://github.com/NicollasZoratto/NicoFlix/commit/cb6cb6c

O que foi feito. Desenvolvimento da aplicação web `app_principal` para listagem e consulta da filmografia do ator Tom Hanks. O backend consome dados dinâmicos da API The Movie Database (TMDB) e

gerencia a persistência de comentários, favoritos e preferências dos usuários através do banco de dados relacional MariaDB.

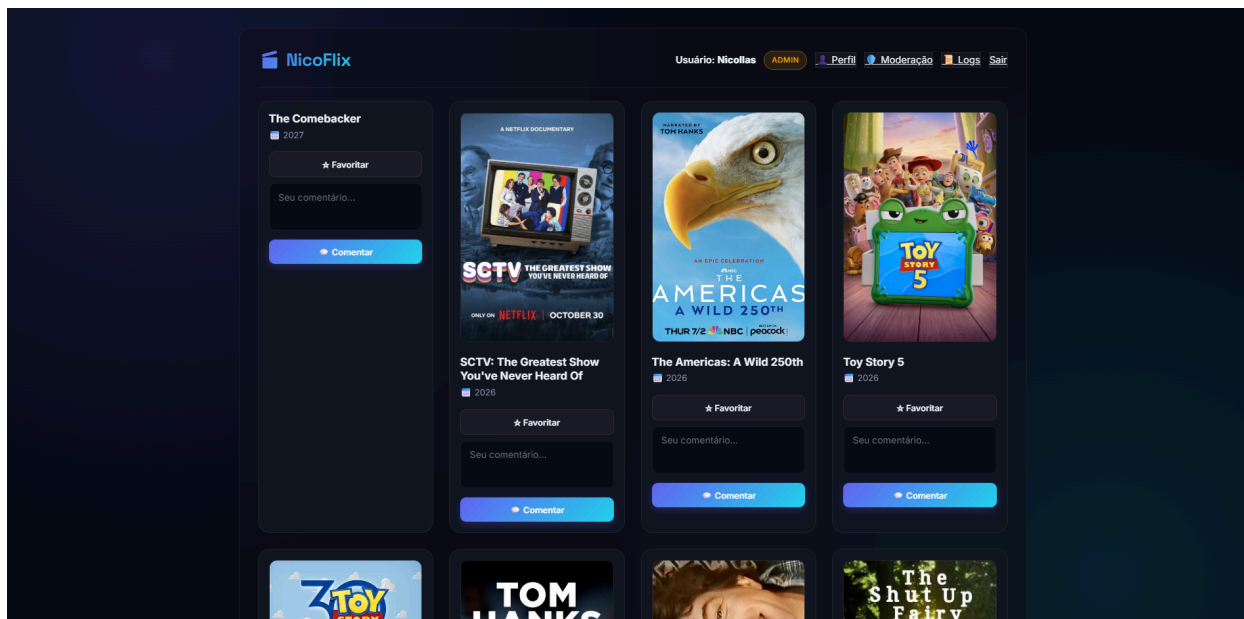


Figura 3: Atividade 2 — commit inicial e estrutura do projeto no GitHub

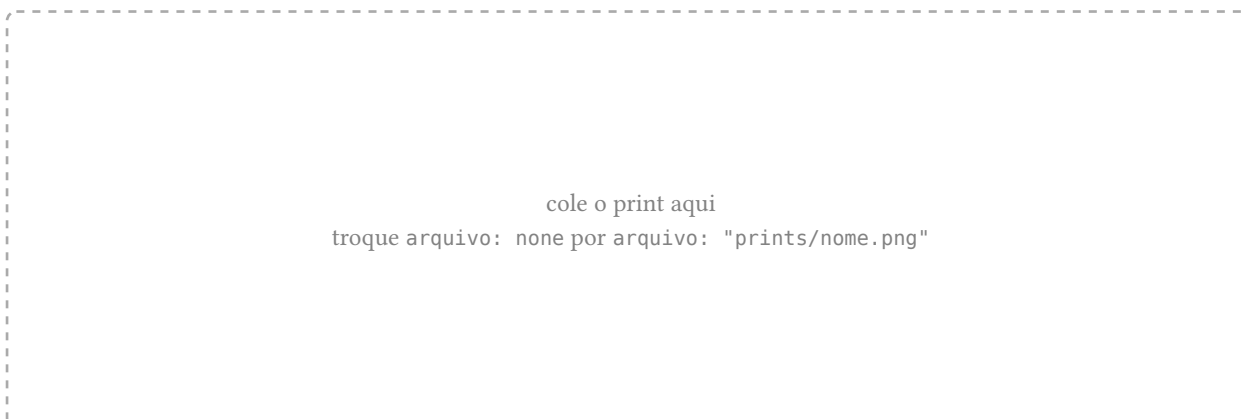


Figura 4: Atividade 2 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. O principal obstáculo esteve na integração inicial do Flask com o banco MariaDB e na manipulação dos objetos JSON retornados pela API externa. A solução envolveu a criação de rotas específicas e isolamento das chamadas da API externa via variáveis de ambiente.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	31/08/2026 19:46
Situação	entregue
Evidência	GitHub — commit cb6cb6c (refactor: substitui projeto antigo pela nova versao completa)
Link	https://github.com/NicollasZoratto/NicoFlix/commit/cb6cb6c

O que foi feito. Separação das responsabilidades de autenticação da aplicação principal (web) para um microserviço independente (auth_service). O auth_service roda de forma privada na rede interna do Docker (sem exposição de portas externas), sendo responsável pelo cadastro, validação de tokens JWT, alteração de credenciais e fluxo de recuperação de senha com tokens temporários.

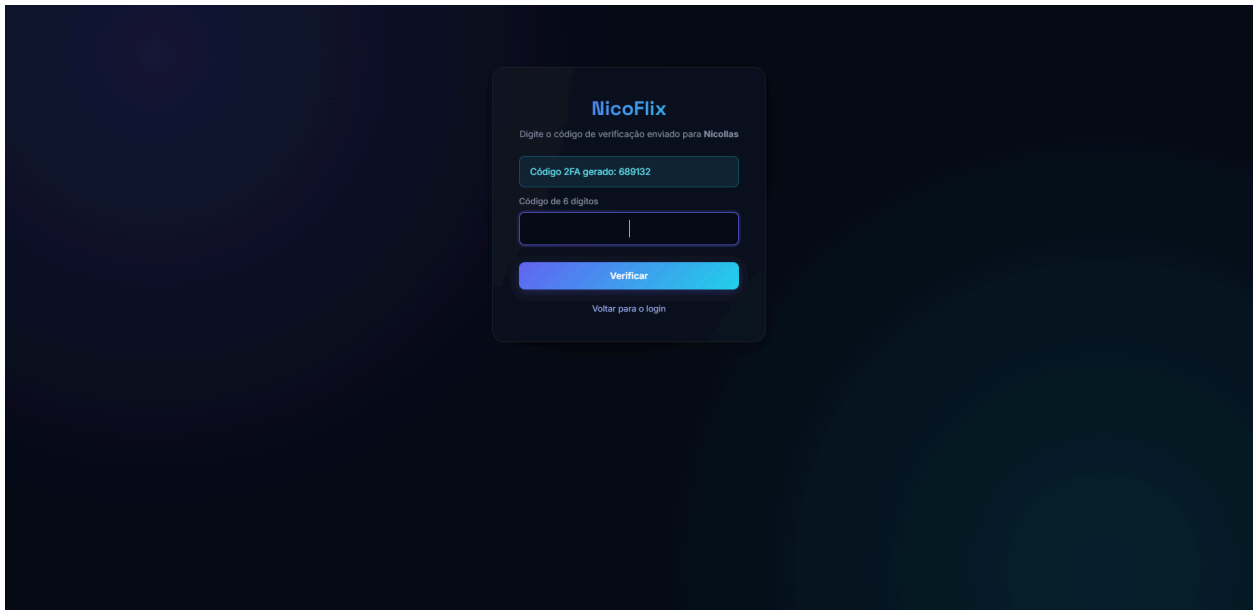


Figura 5: Atividade 3 — configuração do docker-compose.yml e rede interna do auth_service

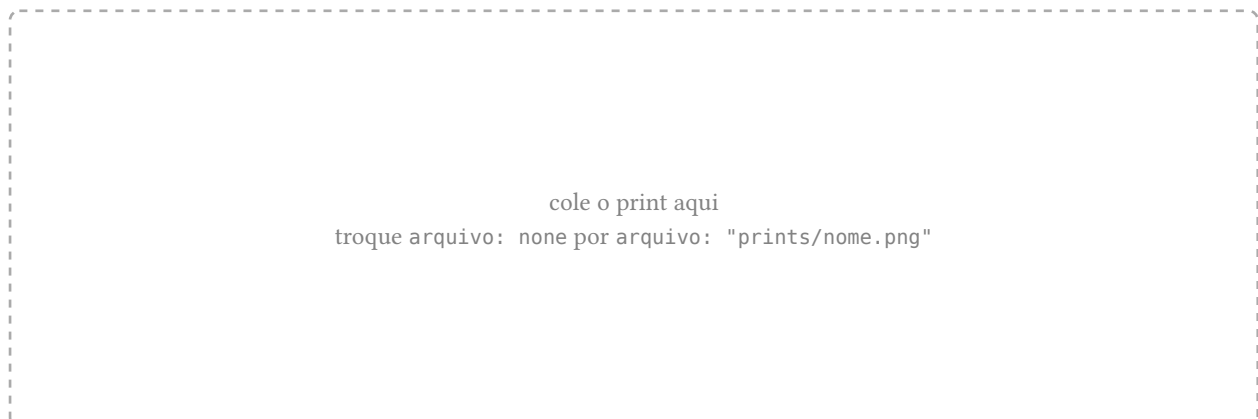


Figura 6: Atividade 3 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. O desacoplamento gerou conflitos de repositório locais devido à migração de código e rotas entre múltiplos diretórios. A dificuldade foi resolvida refatorando a estrutura para o formato multi-container com docker-compose.yml e redefinindo a comunicação HTTP interna entre os serviços (http://auth_service:5001).

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	31/08/2026 22:15
Situação	entregue

Evidência	GitHub — commit 3690909 e container nicoflix_auth no Portainer
Link	https://github.com/NicollasZoratto/NicoFlix/commit/cb6cb6c

O que foi feito. Implementação da arquitetura de Controle de Acesso Baseado em Papéis (RBAC). O campo role (atribuindo perfis como admin ou usuario) foi incorporado ao payload dos tokens JWT emitidos pelo auth_service. A aplicação principal passou a validar o perfil em rotas restritas, bloqueando acessos não autorizados com código de status HTTP 403.

Figura 7: Atividade 4 — código de validação de papéis e retorno HTTP 403 no backend

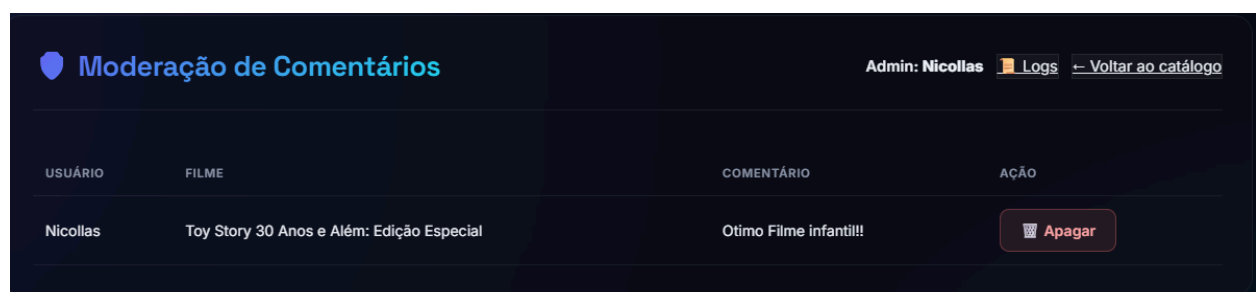


Figura 8: Atividade 4 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. Garantir que a verificação de permissões ocorresse de forma transparente em todas as requisições protegidas. A solução adotada foi criar decorators customizados no Flask que interpretam as alegações (**claims**) do JWT antes de executar as funções de rota.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	23/09/2026 20:05
Situação	entregue
Evidência	GitHub — commit 3690909 (Atividade 5: log-service, RBAC, redesign)

Link

<https://github.com/NicollasZoratto/NicoFlix/commit/3690909>

O que foi feito. Criação do microsserviço `log_service` integrado a um container Redis para armazenamento veloz e temporário de eventos. O serviço registra acessos, operações de auditoria, alterações de privilégios e tentativas de acesso negado, disponibilizando um painel administrativo exclusivo (`admin_logs.html`) para visualização por administradores.

```

1  # Copie este arquivo para ".env" (que fica fora do Git) e ajuste os valores.
2  # O docker-compose.yml lê essas variáveis automaticamente.
3
4  # --- Banco de dados (MariaDB / MySQL remoto) ---
5  DB_HOST=35-226-64-52
6  DB_USER=iac_2026_nicollas_carvalho
7  DB_PASS=nicollas
8  DB_NAME=iac_2026_nicollas_carvalho
9
10 # --- Segredos de aplicação ---
11 JWT_SECRET=chave_secreta_jwt_super_segura
12 SECRET_KEY=chave_secreta_padrao_desenv
13
14 # --- THUMB ---
15 THUMB_API_KEY=6566259ba55415e75fcdac316a8be7
16
17 # --- URL pública do catálogo (usada no link de "Esqueci minha senha") ---
18 PUBLIC_APP_URL=http://localhost:8223
19
20 # --- SMTP / Mailtrap (obrigatório para o e-mail de redefinição de senha
21 # realmente ser enviado; sem isso, o link só aparece no log do container) ---
22 # 1. Crie uma conta gratuita em https://mailtrap.io
23 # 2. Vá em Email Testing > Inboxes > (sua inbox) > SMTP Settings
24 # 3. Copie host/port/receiver/secret da seção "SMTP" em "Inboxes"

```

Figura 9: Atividade 5 — painel `admin_logs.html` e integração com Redis

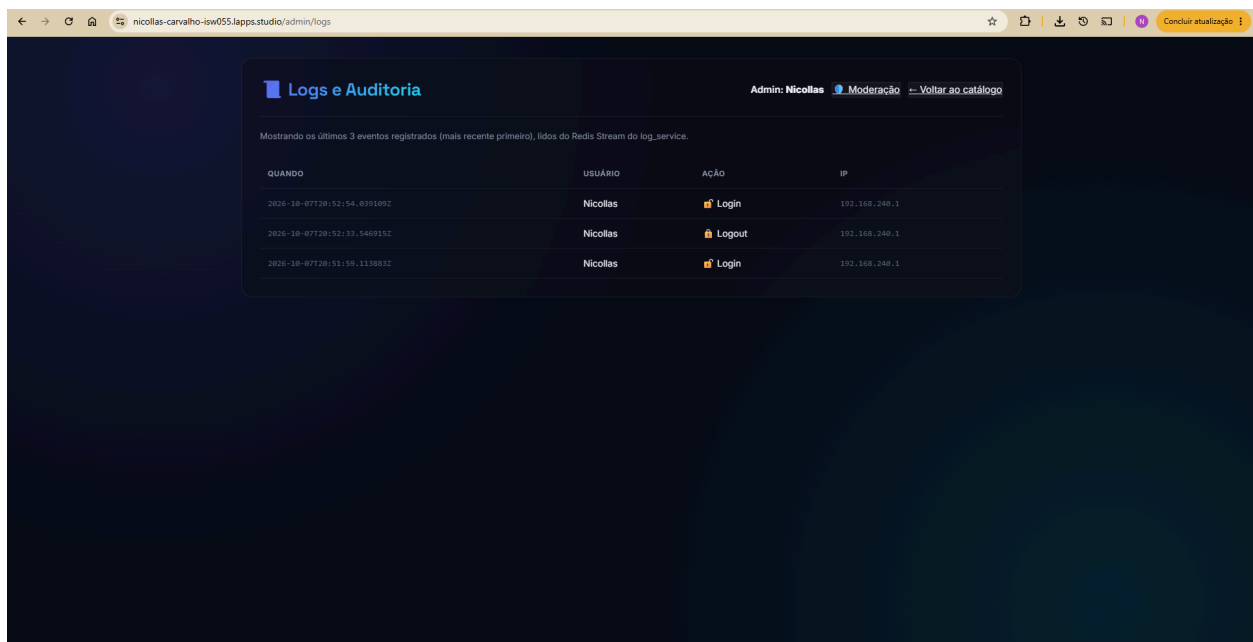


Figura 10: Atividade 5 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. Estabelecer a comunicação assíncrona entre o microsserviço principal e o `log_service` sem comprometer a latência da resposta do usuário. A questão foi solucionada com requisições HTTP internas direcionadas ao serviço de log e gerenciamento de conexões com o banco Redis.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	05/10/2026 19:19
Situação	entregue com atraso
Evidência	GitHub — commit 462bc91 (Atividade 6: perfil de usuario e upload no MinIO)
Link	https://github.com/NicollasZoratto/NicoFlix/commit/462bc91

O que foi feito. Desenvolvimento da página de perfil do usuário com funcionalidade de envio de foto de exibição (avatar). A aplicação conecta-se ao servidor de Object Storage MinIO (MINIO_BUCKET=nicoflix-perfis), armazenando o arquivo físico de imagem no bucket e salvando apenas o caminho e a referência no banco relacional.

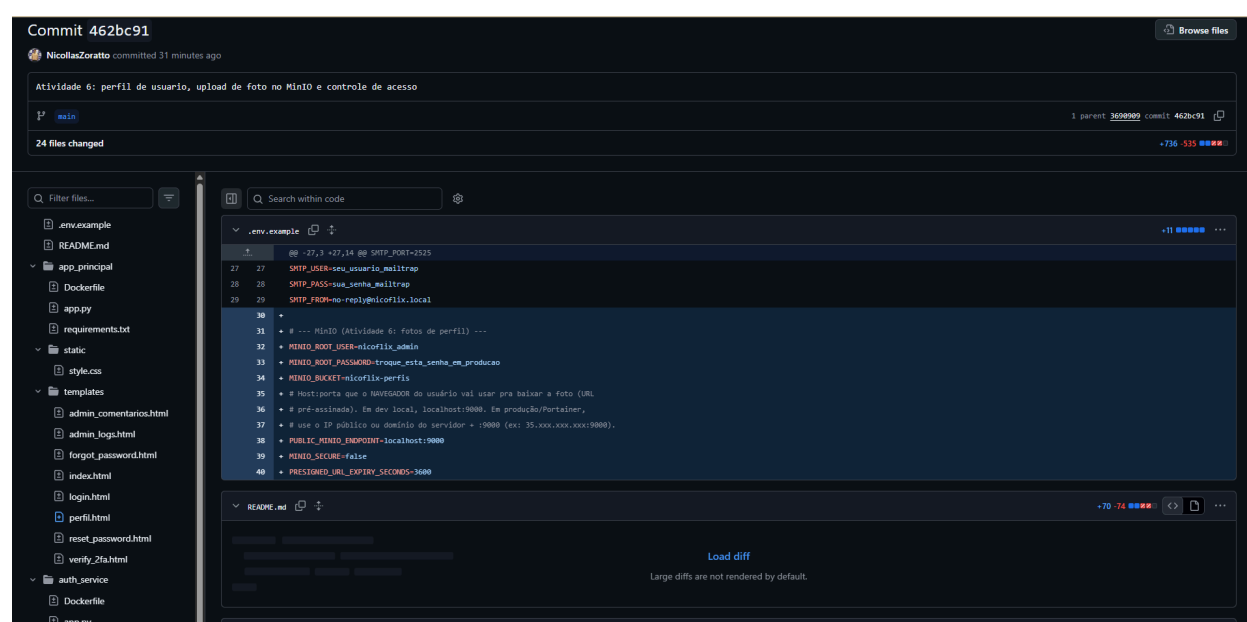


Figura 11: Atividade 6 — upload de avatar funcionando e perfil do usuário

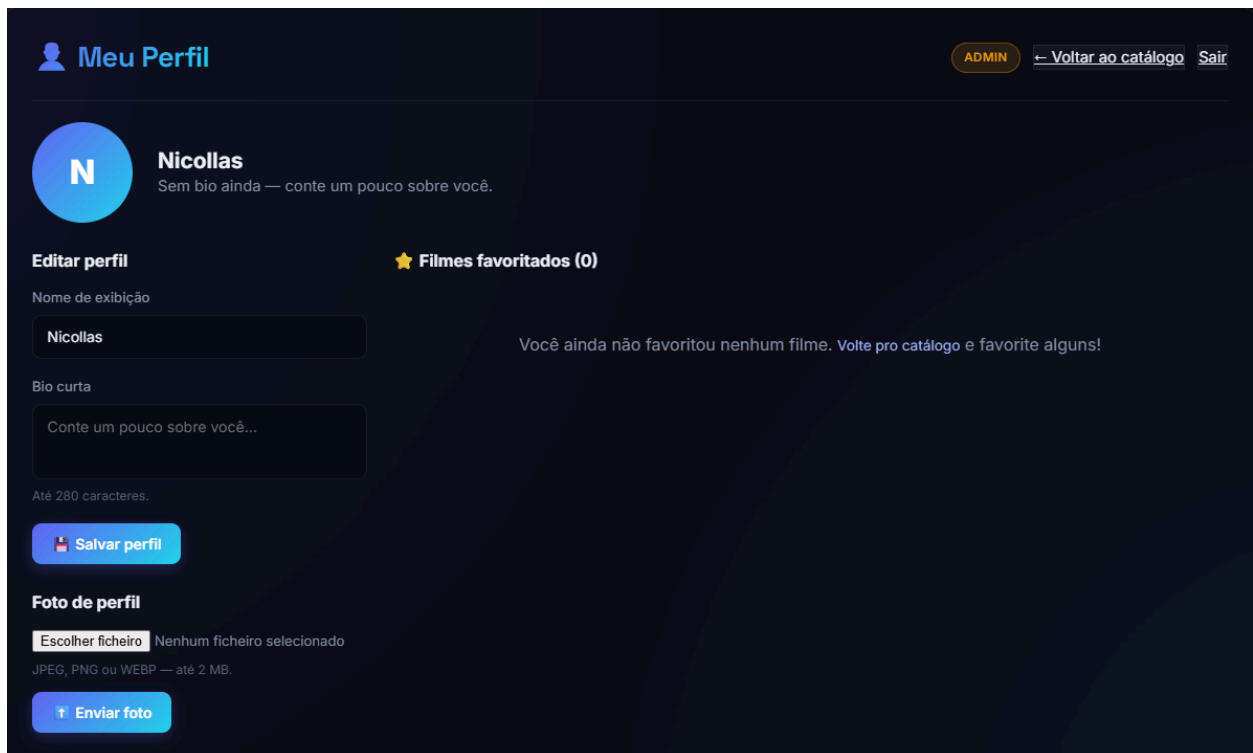


Figura 12: Atividade 6 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. A entrega ocorreu com um pequeno atraso devido à complexidade de configurar as variáveis de ambiente (`MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`, geração de URLs pré-assinadas) e ajustar a resolução do endpoint público do MinIO para exibição das imagens nos navegadores das máquinas clientes. O problema foi solucionado parametrizando o `PUBLIC_MINIO_ENDPOINT` no arquivo `.env`.

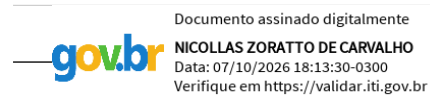
5 Considerações finais

A realização do projeto **NicoFlix** ao longo deste primeiro bimestre proporcionou um aprendizado prático e profundo sobre os pilares da computação em nuvem e a transição de sistemas monolíticos para microserviços. O contato direto com ferramentas de mercado — Docker, Docker Compose, Portainer, Redis, MinIO e JWT — permitiu compreender na prática as vantagens da modularização, do isolamento de redes e do gerenciamento de variáveis de ambiente em infraestruturas modernas.

As maiores dificuldades encontradas concentraram-se na gestão do versionamento ao reestruturar serviços e na configuração adequada da comunicação de rede interna entre os containers. Como aprendizado e melhoria para o próximo bimestre, fica clara a importância de manter mensagens de commit granulares desde o início do ciclo de vida do desenvolvimento, evitando refatorações que exijam a recriação do histórico Git.

6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.



Nicollas Zoratto de Carvalho

Pompeia, 07/10/2026